



UNICAMP



Aula 1: De LLMs a Agentes de IA

[AI Agents in Deep 1]

Marcelo S. Reis [†]

[†] Departamento de Sistemas de Informação
Instituto de Computação, Universidade Estadual de Campinas

Campinas, 25 de agosto de 2026

Sumário

Plano de Desenvolvimento da Disciplina

O que é um Agente de IA?

Engenharia de Controle (*Harness Engineering*)

Projeto Prático: Primeiros Passos

Plano de Desenvolvimento da Disciplina

Documento contendo as instruções da disciplina no nosso Google Classroom, em “Avisos > Plano de Desenvolvimento da Disciplina”

O que é um agente de IA?

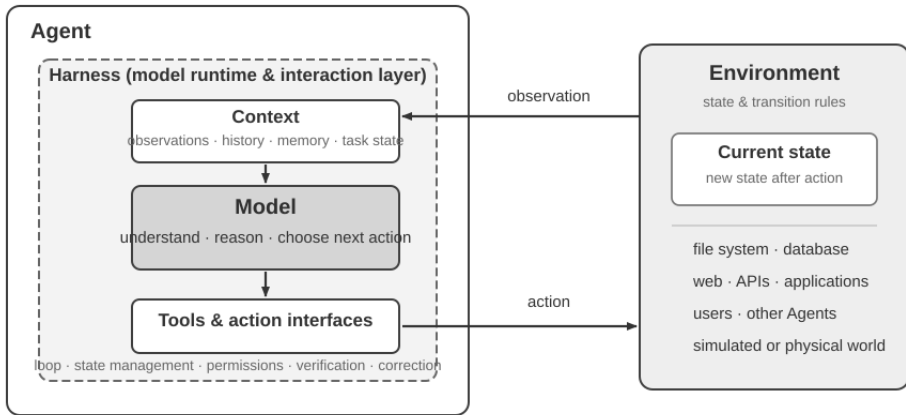
Agente = LLM + Contexto + Ferramentas + Laço de controle

LLM: raciocínio e linguagem. Contexto: informação para decidir. Ferramentas: capacidades externas. Laço: decidir, agir, observar e continuar ou finalizar

LLM versus agente

LLM: entrada → inferência → saída

Agente: observa contexto → decide → chama ferramenta → observa resultado → continua ou finaliza



Intuição	Componente do Agente	Conceito de RL	Papel
Mecanismo de Raciocínio	LLM	Política	A lógica de tomada de decisão que determina “o que fazer a seguir” — dada a informação atual, escolhe a ação mais apropriada dentre todas as opções disponíveis.
Contexto de Trabalho	Construção de contexto	Observações e histórico	Organiza as observações do ambiente e o histórico existente nas informações necessárias para a decisão atual.
Interfaces de Ação	Interfaces de ferramentas	Interfaces de observação/ação	Define quais observações o agente pode ler, quais ações ele pode executar e o formato dessas interfaces.

Tabela: Correspondência entre componentes de um agente e conceitos de aprendizado por reforço.

Exemplos de produtos de agentes, seus contextos de trabalho, interfaces de ação e estratégias

Produto de Agente	Contexto de Trabalho	Interfaces de Ação	Estratégia
Agentes de Programação (ex.: Cursor)	Documentos de requisitos, base de código e ambiente de terminal	Aberta (raciocínio interno, busca de código, leitura/escrita de arquivos, execução de comandos etc.)	Desenvolvimento incremental: entender os requisitos → buscar o código relevante → editar o código → testar e verificar → depurar e corrigir.
Agentes de Busca (ex.: Deep Research)	Recursos da web, bases de dados acadêmicas e arquivos locais	Aberta (raciocínio interno, consultas de busca, leitura da web, geração de resumos)	Aprofundamento iterativo: ajustar a direção da busca com base nas informações já obtidas e sintetizar gradualmente um relatório completo.
Agentes de Controle de Computador (ex.: Browser Use)	Tela do computador, páginas do navegador e sistema de arquivos	Aberta (raciocínio interno, cliques, digitação, rolagem, capturas de tela, execução de código etc.)	Percepção visual + operação: observar a tela → identificar os elementos-alvo → executar ações → verificar os resultados.

Exemplos de produtos de agentes, seus contextos de trabalho, interfaces de ação e estratégias (cont.)

Produto de Agente	Contexto de Trabalho	Interfaces de Ação	Estratégia
Agentes Assistentes de Telefone (ex.: Doubao)	Tela do telefone e aplicativos instalados	Aberta (raciocínio interno, cliques, deslizes, digitação, abertura de aplicativos etc.)	Compreensão de intenção + controle de aplicativos: entender as necessidades do usuário → localizar o aplicativo-alvo → executar as ações → confirmar a conclusão.
Agentes para Tarefas Pessoais (ex.: Pine AI)	Informações da conta do usuário, contas anteriores e base de conhecimento dos provedores de serviço	Aberta (raciocínio interno, realização de chamadas, envio de e-mails, preenchimento de formulários e confirmação com o usuário)	Execução de tarefas em múltiplas etapas: coletar informações → formular uma estratégia de negociação → entrar em contato com o provedor de serviço → negociar → relatar os resultados.

LLM: o mecanismo de raciocínio do Agente

- ❖ O LLM é o núcleo de tomada de decisão do Agente
- ❖ Dada uma solicitação do usuário, o modelo precisa primeiro inferir a **intenção real** por trás do pedido
- ❖ Em seguida, transforma tarefas vagas ou complexas em **etapas executáveis**
- ❖ Durante a execução, decide continuamente:
 - o que fazer em seguida
 - se deve utilizar uma ferramenta (e qual utilizar, com quais argumentos)
- ❖ Essa capacidade de **entender, planejar e executar** deriva do conhecimento adquirido durante o pré-treinamento.

Raciocínio interno

- ❖ Uma capacidade distintiva dos Agentes baseados em LLM é o **raciocínio interno**
- ❖ Antes de agir, o Agente pode **planejar e raciocinar sobre a tarefa**
 - Esse raciocínio não altera diretamente o ambiente externo, mas pode melhorar significativamente as ações subsequentes
- ❖ O modelo aprende padrões de raciocínio presentes no conhecimento humano, incluindo:
 - leis matemáticas
 - relações causais
 - estratégias de decomposição de problemas
- ❖ Assim, diferentemente de Agentes tradicionais de aprendizado por reforço, Agentes baseados em LLM podem raciocinar sobre um **corpo estruturado de conhecimento**, em vez de depender de tentativa e erro cego

Quando o modelo se torna o produto

- ❖ O paradigma **Model as Agent** representa uma direção recente no desenvolvimento de Agentes de IA
- ❖ Modelos avançados podem internalizar o uso de ferramentas como uma capacidade nativa por meio do **pós-treinamento**, especialmente aprendizado por reforço
- ❖ O próprio modelo decide pelo uso de ferramentas
- ❖ Isso reduz a necessidade de **orquestração manual** do fluxo de execução
- ❖ Porém, isso não torna a camada de infraestrutura menos importante. Pelo contrário: **quanto mais forte o modelo, maior a importância do Harness** que o envolve

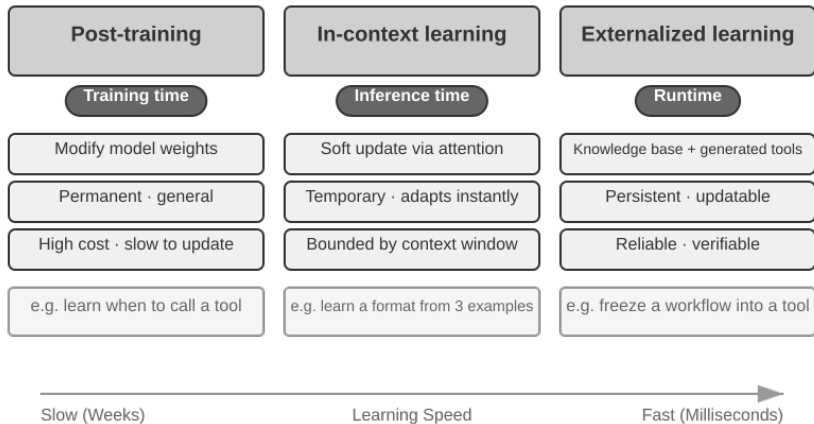
Harness: canalizando a capacidade do modelo

- ❖ O Harness inclui mecanismos de:
 - gerenciamento de contexto
 - interfaces de ferramentas
 - restrições de segurança
 - verificação e correção
- ❖ Quanto maior a autoridade de decisão concedida ao modelo, maior o impacto potencial de uma decisão incorreta
- ❖ Portanto, modelos mais capazes exigem mecanismos mais refinados de **restrição, verificação e correção**

Mecanismos de aprendizado do Agente

- ❖ Mudanças no comportamento de um Agente não acontecem apenas durante o treinamento do modelo
- ❖ Dependendo de **onde** ocorre a atualização e de **quanto tempo** ela persiste, podemos distinguir três mecanismos:
 1. **Adaptação contextual** dentro da tarefa
 2. **Atualização de artefatos externos** entre tarefas
 3. **Atualização dos parâmetros** durante ciclos de treinamento
- ❖ Os três mecanismos não são mutuamente exclusivos: eles operam em **diferentes escalas de tempo** e podem ser combinados

Três níveis de atualização das capacidades



Adaptação contextual

- ❖ A **adaptação contextual** ocorre durante a tarefa atual
- ❖ Exemplos, estado da tarefa e resultados de recuperação (*retrieval*) entram no contexto do modelo
- ❖ O modelo pode então **ajustar seu comportamento imediatamente**
- ❖ Entretanto, essa adaptação não altera o estado persistente utilizado na próxima sessão
- ❖ **Vantagens:** velocidade e baixo custo
- ❖ **Limitações:** tamanho da janela de contexto e organização das informações

Atualização de artefatos externos

- ❖ Para que mudanças persistam entre tarefas, o sistema pode atualizar **artefatos externos**.
- ❖ **Conhecimento**: fatos e experiências podem ser organizados em documentos.
- ❖ **Estratégias**: comportamentos expressáveis em linguagem podem ser registrados em Prompts ou Skills.
- ❖ **Procedimentos e restrições**: podem ser codificados em programas e Harnesses
- ❖ Esses artefatos são **auditáveis e revisáveis**
- ❖ Porém, o Agente ainda precisa acessá-los durante a execução por meio do contexto ou de interfaces de ferramentas

Atualização dos parâmetros do modelo

- ❖ Algumas capacidades são difíceis de expressar por meio de regras externas
- ❖ Nesses casos, pode ser necessário atualizar os **parâmetros do modelo** por meio do pós-treinamento
- ❖ Essa abordagem possui **maior custo de implantação**
- ❖ Em contrapartida, pode produzir uma generalização mais ampla e natural das capacidades aprendidas
- ❖ Assim, os três mecanismos podem ser vistos como uma hierarquia:
Contexto → adaptação imediata **Artefatos** → acumulação controlada
Parâmetros → internalização

Harness Engineering: além do modelo

- ❖ Um Agente combina **LLM + Contexto + Ferramentas**
- ❖ O mecanismo básico funciona, mas é **frágil**
- ❖ O modelo pode:
 - alucinar ferramentas ou parâmetros
 - escolher a ferramenta errada
 - não conseguir se recuperar de erros
- ❖ Existe uma grande lacuna entre uma demonstração funcional e um produto confiável
- ❖ **Harness Engineering** existe para fechar essa lacuna.

Agent = Model + Harness

- ❖ A fórmula original descreve a **composição interna** do Agente:

$$\text{Agent} = \text{LLM} + \text{Context} + \text{Tools}$$

- ❖ O Harness fornece uma visão de **implementação**:

$$\text{Agent} = \text{Model} + \text{Harness}$$

- ❖ O Harness contém o contexto, as ferramentas e as camadas de proteção
- ❖ As duas visões descrevem o **mesmo sistema em níveis diferentes** de abstração

A composição de um Agente em produção

❖ O Harness é construído em torno de cinco elementos:

1. **Context** — informação relevante
2. **Tools** — interfaces de ação
3. **Constrain** — limites de comportamento
4. **Verify** — verificação dos resultados
5. **Correct** — recuperação de falhas

❖ A composição completa é:

$$\text{Agent} = \text{Model} + \text{Harness}$$

$$\text{Harness} = \text{Context} + \text{Tools} + \text{Constrain} + \text{Verify} + \text{Correct}$$

❖ O Agente interage continuamente com o ambiente:

$$\text{Agent} \leftrightarrow \text{Environment}$$

O Harness fica entre o modelo e o ambiente

- ❖ O Harness **não é tudo que está fora do modelo**: ele é a camada de **runtime e governança** dentro da fronteira do Agente
- ❖ **Harness**: interfaces de ferramentas, adaptadores, permissões, sandbox e mecanismos de reset
- ❖ **Environment**: arquivos, processos, bancos de dados, páginas web, usuários e mundo físico
- ❖ O Harness **faz a intermediação** da interação entre Modelo e Ambiente

Cinco elementos do Harness

- ❖ **Context:** fornecer informação suficiente para cada decisão
- ❖ **Tools:** fornecer interfaces claras para agir sobre o ambiente
- ❖ **Constrain:** definir o que o Agente pode ou não pode fazer
- ❖ **Verify:** verificar automaticamente se a ação funcionou
- ❖ **Correct:** recuperar, repetir ou reverter ações incorretas

O loop de controle

- ❖ O Agente observa o ambiente
- ❖ O Harness constrói o contexto
- ❖ O modelo escolhe ações
- ❖ O Harness restringe as ações permitidas
- ❖ O ambiente executa as ações e produz novas observações
- ❖ O Harness verifica o resultado e corrige falhas

Observe → Reason → Constrain → Act → Verify → Correct

Completando tarefas com confiabilidade

- ❖ **Context + Tools** permitem que o Agente execute tarefas
- ❖ **Constrain + Verify + Correct** tornam essa execução confiável e segura
- ❖ Frameworks iniciais enfatizavam:

Context + Tools

- ❖ Sistemas de produção deslocam o foco para:

Constrain + Verify + Correct

- ❖ A vantagem competitiva passa da **execução** para a **execução confiável**

De Prompt Engineering a Graph Engineering

- ❖ **Prompt Engineering**

Otimiza as instruções fornecidas ao modelo

- ❖ **Context Engineering**

Gerencia todas as informações disponíveis ao modelo

- ❖ **Harness Engineering**

Controla o sistema em torno do modelo

- ❖ **Loop Engineering**

Organiza operação autônoma sustentada entre execuções

- ❖ **Graph Engineering**

Organiza loops, programas e aprovações em um grafo explícito de execução

Princípios para construir Agentes eficazes

- ❖ **“Keep it simple”**

Comece pela solução mais simples; adicione complexidade apenas quando necessário

- ❖ **“Keep it transparent”**

Torne planejamento, execução e trajetória de decisões observáveis

- ❖ **Desenhe uma boa ACI (Agent-Computer Interface)**

Projete a interface de ferramentas a partir da perspectiva do Agente

- ❖ **“Design errors out”**

Sempre que possível, torne o erro impossível por construção (filosofia *poka-yoke*, adotada pela montadora Toyota)

Como escolher um modelo

- ❖ O modelo é a **base da inteligência** do Agente
- ❖ Não escolha apenas pelo leaderboard: **avalie no seu próprio problema**
- ❖ **Modelos fechados**: maior capacidade, mas maior custo e dependência do fornecedor
- ❖ **Modelos abertos**: menor custo, implantação privada e possibilidade de fine-tuning
- ❖ Verifique também as **políticas de uso** e limites do fornecedor
- ❖ Para tarefas complexas, prefira modelos com **capacidade de raciocínio**

Outros critérios de seleção

- ❖ **Velocidade de geração**

Agentes realizam muitas rodadas de inferência: a latência se acumula

- ❖ **Capacidade multimodal**

Imagens, áudio e vídeo podem ser requisitos essenciais

- ❖ **Tool calling**

Modelos variam significativamente na capacidade de utilizar ferramentas

- ❖ **Fallback**

Para tarefas críticas, considere transferi-las para um humano ou para outro modelo mais compatível com elas

Workflow vs. Agente Autônomo

- ❖ A **orquestração** determina **como o contexto e as ferramentas fluem** entre as chamadas ao modelo
- ❖ Comece pelo mais simples:
 1. uma chamada LLM
 2. workflow
 3. Agente autônomo
- ❖ Use **workflow** quando as etapas são previsíveis
- ❖ Use **Agente autônomo** quando decisões e caminhos são imprevisíveis
- ❖ Autonomia normalmente troca **latência e custo** por maior capacidade de resolver tarefas complexas

Workflow: orquestração determinística

- ❖ Um workflow define antecipadamente o **caminho de execução**
- ❖ O código determina a sequência; o LLM atua dentro de cada etapa.
- ❖ Exemplo: reserva de voo
 1. verificar identidade
 2. buscar voos
 3. realizar pagamento
 4. confirmar reserva
- ❖ **Vantagem:** controle rigoroso do processo
- ❖ **Limitação:** baixa flexibilidade diante de eventos imprevistos

Workflow como camada de adaptação

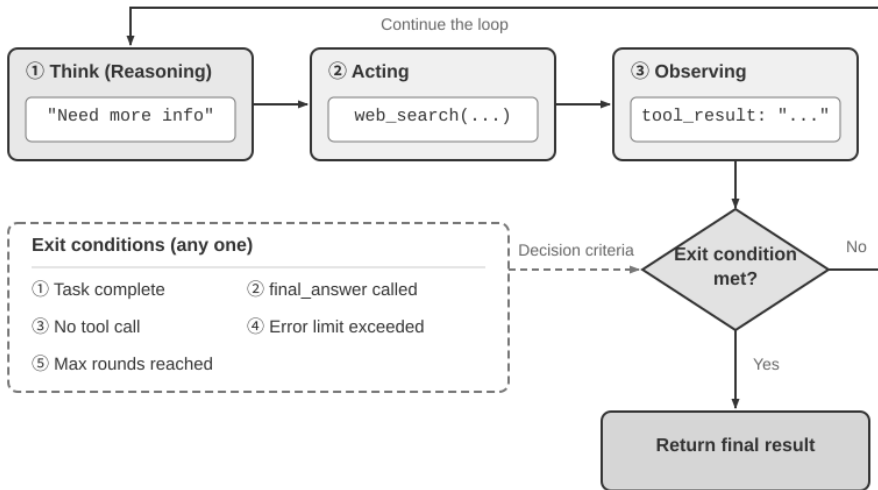
- ❖ Exemplo: geração de imagens com um modelo que exige prompts específicos
- ❖ O workflow pode ter dois nós:
 1. **Prompt Rewriting**
 2. **Image Generation**
- ❖ O LLM atua como um **adaptador**: traduz a linguagem humana para o formato esperado pela ferramenta
- ❖ Quando o modelo passa a gerar imagens nativamente, o adaptador pode se tornar desnecessário
- ❖ **Modelos mais capazes absorvem gradualmente camadas do Harness**

Agente autônomo: decisão em tempo real

- ❖ O caminho de execução **não é definido previamente**
- ❖ O Agente decide seus próximos passos com base no **feedback do ambiente**
- ❖ Exemplo: reserva de voo
 - buscar voos
 - descobrir necessidade de login
 - verificar identidade
 - retomar a busca
 - adaptar critérios conforme a resposta do usuário
- ❖ O Agente precisa saber **planejar, detectar falhas e mudar de estratégia**
- ❖ Autonomia exige **condições explícitas de parada**

Loop de um Agente autônomo

- ❖ O Agente é essencialmente um **LLM usando ferramentas em um loop**.
- ❖ A cada rodada:
 1. observa o ambiente
 2. raciocina
 3. escolhe ações
 4. executa
 5. recebe feedback
- ❖ Condições de saída:
 - tarefa concluída
 - resposta final sem tool call
 - erro irrecoverável
 - máximo de iterações atingido



Quando usar Agentes autônomos?

- ❖ São adequados para problemas **abertos**, cujo número de etapas não é conhecido previamente
- ❖ Exemplos:
 - Coding Agents
 - Computer Use
 - pesquisa iterativa
- ❖ A autonomia aumenta a capacidade de adaptação, mas também aumenta:
 - custo
 - latência
 - propagação de erros
- ❖ Portanto, exige **sandbox, guardrails, monitoramento e human-in-the-loop** em pontos críticos

Ideia central

- ❖ Modelos cada vez mais capazes reduzem a necessidade de alguns componentes de adaptação
- ❖ Mas sistemas de produção continuam precisando de **Harness Engineering**
- ❖ O desafio deixa de ser apenas:
“O Agente consegue realizar a tarefa?”
- ❖ E passa a ser:
“O Agente consegue realizá-la de forma confiável?”
- ❖ **A vantagem competitiva migra para a engenharia ao redor do modelo!**

Projeto Prático: Primeiros Passos

- ❖ Enunciado do Entregável 1, cujo prazo de entrega é **6 de setembro**, será liberado nesta quinta-feira (27/08)
- ❖ Tarefas para esta semana:
 1. Definir o grupo (duplas ou trios)
 2. Definir um tema/problema

1. Definir o grupo

- ❖ Projeto deverá ser executado em duplas ou trios
- ❖ Quem estiver em dificuldades em encontrar grupo, contactar o Renan (monitor), que ele auxiliará nesse processo
- ❖ Cada grupo formado deverá enviar, no Google Classroom, um pdf de uma página, contendo os nomes completos dos componentes do grupo e também uma breve descrição do problema/tema escolhido
 - Prazo de entrega: **1 de setembro**
 - Esta entrega **não** vale nota

2. Definir um tema/problema

❖ Algumas sugestões de temas:

- Análise da literatura científica
- Desenvolvimento de software
- Análise de dados
- Tutoria educacional
- Análise de documentos
- Planejamento de viagens
- Pesquisa de mercado
- Pesquisa histórica

❖ Todavia, vocês são livres para trabalharem com o tema que preferirem. Outras sugestões estão disponíveis em: github.com/ashishpatel26/500-AI-Agents-Projects